Lab Handout 1-C
Serial Peripheral Interface (SPI) Bring-Up: STM32 ↔ SX1276 Register Access

Capstone — Module 1: Radio Frequency (RF) Hardware Foundations

1 Objective

Establish reliable SPI communication between the STM32 Nucleo-L476RG and the SX1276-based RFM95W module. You will read the `RegVersion` register and receive the expected value `0x12`, then demonstrate write/read-back and radio mode transitions. This handout is the first point at which firmware actually touches the RF silicon.

2 Prerequisites

- Lab Handout 1-A complete (breakout physically assembled).
- Lab Handout 1-B complete (toolchain working, blinky verified).

3 Materials

- Assembled RFM95W breakout (from LH1-A).
- Nucleo-L476RG + USB cable (from LH1-B).
- Half-size breadboard, 8–10 male–female jumper wires.
- Logic analyzer (optional but strongly recommended for troubleshooting).

4 Wiring

RFM95W Pin	Function	Nucleo Pin	Board Label
`VIN`	+3.3 V supply	`3V3`	3V3
`GND`	Ground	`GND`	GND
`SCK`	SPI clock	`PA5`	SCK/D13
`MISO`	SPI data in (to Microcontroller Unit (MCU))	`PA6`	MISO/D12
`MOSI`	SPI data out (from MCU)	`PA7`	PWM/MOSI/D11
`CS`	Chip select (active low)	`PB6`	PWM/CS/D10
`RST`	Reset (active low)	`PA9`	D8
`G0` (`DIO0`)	Packet-done interrupt	`PA8`	D7

Board Label refers to the silkscreen text on the Nucleo CN10 connector. Locate pins by this label; the STM32 pin name (PA5, etc.) is the CubeMX reference.

[Figure placeholder]

`LH1C_wiring_diagram.png`

Schematic of the Nucleo-L476RG ↔ RFM95W wiring. Power, ground, full-duplex SPI1, chip-select, reset, and `DIO0` interrupt line are all shown.

Figure 1: Schematic of the Nucleo-L476RG ↔ RFM95W wiring. Power, ground, full-duplex SPI1, chip-select, reset, and `DIO0` interrupt line are all shown.

[Figure placeholder]

`LH1C_breadboard_photo.jpg`

Physical breadboard build matching the schematic above. Keep jumper leads short to minimize SPI ringing.

Figure 2: Physical breadboard build matching the schematic above. Keep jumper leads short to minimize SPI ringing.

Wire Before You Power

Double-check `VIN` vs. `GND`. The RFM95W will tolerate 3.3–5 V on `VIN` (it has an on-board regulator), but swapping `VIN` and `GND` *will* destroy the module. Verify with a DMM before plugging the Nucleo into USB.

5 Procedure

5.1 Step 1 — Configure peripherals via STM32CubeMX

1. Create a new project in STM32CubeMX following the same steps as Lab Handout 1-B (STM icon → Launch STM32CubeMX → Board Selector → NUCLEO-L476RG → Start Project → Project Manager). Name the project **Nucleo_RFM95W_Base** and save it to a local folder outside OneDrive. Set the toolchain to **CMake**. This project will serve as the reusable base configuration for all subsequent RF labs.

2. **Enable SPI1:**

   (a) In the pinout view, click pin `PA5` and select **SPI1_SCK** from the popup. Repeat for `PA6` → **SPI1_MISO** and `PA7` → **SPI1_MOSI**.

   (b) In the left-hand **Categories** tree, expand **Connectivity** and click **SPI1**.

   (c) Set the **Mode** dropdown to **Full-Duplex Master**. The pins assigned above will be confirmed in the pinout view.

3. **Configure SPI1 parameters:**

   (a) With SPI1 still selected, click the **Configuration** tab (or **Parameter Settings** in newer CubeMX versions).

   (b) Set each parameter per Table 1 below. Pay particular attention to CPOL=0 and CPHA=1 Edge — an incorrect clock phase is the most common cause of a failed `RegVersion` read.

   (c) Set **NSS** to **Software** — chip select will be driven manually in firmware.

Table 1: SPI1 configuration for RFM95W register access.

Parameter	Value
Mode	Full-Duplex Master
Data size	8 bits
First bit	MSB first
Prescaler	**32** (yields 2.5 MHz at 80 MHz PCLK2; safe margin below the 10 MHz max)
CPOL	0 (Low)
CPHA	1 Edge
NSS	Software

4. **Configure GPIO pins for CS, RST, and DIO0:**

   (a) In the pinout view, click pin `PB6`. Select **GPIO_Output** from the popup.

   (b) Repeat for `PA9` and `PA8`.

   (c) To assign user labels, click **System Core** → **GPIO** in the categories tree. Select each pin and set its **User Label**: `PB6` → `RFM_CS`, `PA9` → `RFM_RST`, `PA8` → `RFM_DIO0`. These labels become the `_GPIO_Port` and `_Pin` macro names in generated code.

   (d) Leave `PA8` (`RFM_DIO0`) as a plain GPIO output for now; it will be reconfigured as an `EXTI` interrupt input in a later handout.

5. **Enable USART2** (used to send `printf` output to the ST-LINK virtual COM port):
   (a) In the categories tree, expand **Connectivity** and click **USART2**.
   (b) Set **Mode** to **Asynchronous**.
   (c) In **Parameter Settings**, confirm **Baud Rate** is **115200**, **Word Length** 8 bits, **Parity** None, **Stop Bits** 1.
6. Click **Generate Code**. Return to VS Code and run `cube bundle init --project` in the terminal if prompted.

[Figure placeholder]

`LH1C_spi_config.png`

STM32CubeMX SPI1 parameter panel configured per Table 1: 8-bit data, MSB first, CPOL=0, CPHA=1 Edge, prescaler 32.

Figure 3: STM32CubeMX SPI1 parameter panel configured per Table 1: 8-bit data, MSB first, CPOL=0, CPHA=1 Edge, prescaler 32.

5.2 Step 2 — Redirect `printf` to USART2

1. In `main.c`, locate the `/* USER CODE BEGIN Includes */` section near the top of the file and add:

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
/* USER CODE END Includes */
```

2. Locate the `/* USER CODE BEGIN PFP */` section and add prototypes for every function you will define in this lab:

```
/* USER CODE BEGIN PFP */
int __io_putchar(int ch);
uint8_t rfm_read_reg(uint8_t addr);
void rfm_write_reg(uint8_t addr, uint8_t val);
/* USER CODE END PFP */
```

3. Scroll to the `/* USER CODE BEGIN 4 */` section near the bottom of the file and add the definition:

```
/* USER CODE BEGIN 4 */
int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart2, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
    return ch;
}
/* USER CODE END 4 */
```

Keeping the prototype and definition inside the `USER CODE` comment blocks ensures they are preserved if you regenerate code from CubeMX.

5.3 Step 3 — Write a minimal register-read function

Implement the following helper:

```
uint8_t rfm_read_reg(uint8_t addr) {
    uint8_t tx[2] = { addr & 0x7F, 0x00 };  // bit 7 = 0 => read
    uint8_t rx[2] = { 0, 0 };
    HAL_GPIO_WritePin(RFM_CS_GPIO_Port, RFM_CS_Pin, GPIO_PIN_RESET);
    HAL_SPI_TransmitReceive(&hspi1, tx, rx, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(RFM_CS_GPIO_Port, RFM_CS_Pin, GPIO_PIN_SET);
    return rx[1];
}
```

Add the write helper immediately after in the same `USER CODE BEGIN 4` section:

```
void rfm_write_reg(uint8_t addr, uint8_t val) {
    uint8_t tx[2] = { addr | 0x80, val };  // bit 7 = 1 => write
    HAL_GPIO_WritePin(RFM_CS_GPIO_Port, RFM_CS_Pin, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, tx, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(RFM_CS_GPIO_Port, RFM_CS_Pin, GPIO_PIN_SET);
}
```

5.4 Step 4 — Reset the module and read `RegVersion`

1. In `main.c`, locate the `/* USER CODE BEGIN 2 */` section (after all HAL and peripheral initialization). Add the following reset sequence and register read:

```
/* USER CODE BEGIN 2 */

/* Reset the RFM95W */
HAL_GPIO_WritePin(RFM_RST_GPIO_Port, RFM_RST_Pin, GPIO_PIN_RESET);
HAL_Delay(1);  // hold reset low for at least 100 us
HAL_GPIO_WritePin(RFM_RST_GPIO_Port, RFM_RST_Pin, GPIO_PIN_SET);
HAL_Delay(10); // wait for module to come out of reset

/* Read and print the version register */
uint8_t version = rfm_read_reg(0x42);
printf("RegVersion = 0x%02X\r\n", version);

/* USER CODE END 2 */
```

2. Build the project: Command Palette → **CMake: Build**. Confirm zero errors.
3. Flash and run: press `F5`. Once the debugger halts at `main`, press `F5` again to continue.
4. Open the Serial Monitor panel:
   (a) Command Palette (`Ctrl+Shift+P`) → **Serial Monitor: Focus on Serial Monitor View**.
   (b) In the Serial Monitor panel at the bottom of VS Code, click **Open Serial Monitor**.
   (c) Select the **STLink Virtual COM Port** from the port dropdown.
   (d) Set the baud rate to **115200**.
   (e) Click **Start Monitoring**.
5. You should see the following output:

```
RegVersion = 0x12
```

   Any other value indicates a wiring or configuration error — see the Troubleshooting section.

Why 0x12?

The SX1276 silicon revision register returns `0x12` for the V1b mask set used in the RFM9x breakouts. This is the single most reliable sanity check for SPI wiring, clock polarity, and reset sequencing: if you read `0x00` or `0xFF`, the module is not talking to you; if you read anything else, you have a clock phase or byte-order issue.

[Figure placeholder]

`LH1C_regversion_printf.png`

Serial monitor output showing the first successful `RegVersion = 0x12` read. This is the canonical sanity check for the entire SPI stack.

Figure 4: Serial monitor output showing the first successful `RegVersion = 0x12` read. This is the canonical sanity check for the entire SPI stack.

5.5 Step 5 — Register write / read-back

Add the following code in `main.c` inside the `/* USER CODE BEGIN 2 */` block, immediately after the `RegVersion` read from Step 4:

1. First put the module into `SLEEP` mode — the SX1276 requires this before frequency registers can be written:

   ```
   rfm_write_reg(0x01, 0x80);  // LoRa mode, SLEEP
   HAL_Delay(10);
   ```

2. Write the three frequency registers to set the carrier to 906.5 MHz:

   ```
   rfm_write_reg(0x06, 0xD9);  // RegFrfMsb
   rfm_write_reg(0x07, 0x0C);  // RegFrfMid
   rfm_write_reg(0x08, 0xCC);  // RegFrfLsb
   ```

3. Read the same registers back and print each value:

   ```
   printf("FrfMsb = 0x%02X\r\n", rfm_read_reg(0x06));
   printf("FrfMid = 0x%02X\r\n", rfm_read_reg(0x07));
   printf("FrfLsb = 0x%02X\r\n", rfm_read_reg(0x08));
   ```

4. Build, flash (`F5`), and check the serial monitor. The three printed values should match `0xD9`, `0x0C`, `0xCC`. A mismatch usually indicates a CPOL/CPHA error or a floating `MISO` line.

5.6 Step 6 — Mode transitions

The `RegOpMode` register (`0x01`) controls the radio operating state. The low 3 bits select the mode; bit 7 selects LoRa vs. FSK. Add the following to `/* USER CODE BEGIN 2 */` after the Step 5 code, then build and flash to observe each transition in the serial monitor.

1. Read the current mode and print it:

```
printf("OpMode = 0x%02X\r\n", rfm_read_reg(0x01));
```

2. Transition to LoRa `SLEEP` (bits [2:0] = `000`):

```
rfm_write_reg(0x01, 0x80);
HAL_Delay(10);
printf("OpMode (SLEEP)    = 0x%02X\r\n", rfm_read_reg(0x01));
```

3. Transition to LoRa `STANDBY` (bits [2:0] = `001`):

```
rfm_write_reg(0x01, 0x81);
HAL_Delay(10);
printf("OpMode (STANDBY)  = 0x%02X\r\n", rfm_read_reg(0x01));
```

4. Transition to LoRa `TX` (bits [2:0] = `011`):

```
rfm_write_reg(0x01, 0x83);
HAL_Delay(10);
printf("OpMode (TX)       = 0x%02X\r\n", rfm_read_reg(0x01));
```

   The module will return to `STANDBY` automatically after any transmission attempt; seeing `0x81` on a subsequent read is expected.

6 Verification Checklist

- ☐ `RegVersion` reads `0x12`.
- ☐ Write/read-back to `RegFrf*` registers returns identical values.
- ☐ `RegOpMode` mode bits transition correctly through `SLEEP`, `STANDBY`, `TX`.
- ☐ Serial output shows all register values in hex over the ST-LINK virtual COM port.

Troubleshooting

- `RegVersion` **reads** `0x00`: `MISO` not connected, or `CS` never asserted.
- `RegVersion` **reads** `0xFF`: `MISO` floating (pull-up). Check the jumper to `PA6`.
- **Reads a seemingly-random but repeatable value**: CPOL/CPHA wrong, or bit 7 (read/write) flipped in your address byte.
- **First read is wrong, subsequent reads correct**: missing reset delay; increase the post-reset wait to 20 ms.

Lab Handout 1-C Deliverable

1. Source code for your `rfm_read_reg` / `rfm_write_reg` helpers and the `main` test sequence.
2. Serial monitor log showing `RegVersion = 0x12`, frequency register write/read-back, and all three mode transitions.

3. If a logic analyzer was used: one screenshot of a successful `RegVersion` read transaction with `CS`, `SCK`, `MOSI`, `MISO` traces.

Glossary

MCU	Microcontroller Unit	1
RF	Radio Frequency	1
SPI	Serial Peripheral Interface	1, 2, 3, 4, 6, 7